

CI/CD Pipeline with Vulnerability Mitigation

Nalini M K

Assistant Professor, Department of
Information Science and Engineering
B.M.S. College of Engineering,
Bangalore, India.
nalini.ise@bmsce.ac.in

Mahalakshmi B S

Assistant Professor, Department of
Information Science and Engineering
B.M.S. College of Engineering,
Bangalore, India.
bsmahalakshmi.ise@bmsce.ac.in

Nevya Khandelwal

Department of Information Science and
Engineering
B.M.S. College of Engineering,
Bangalore, India.
nevyas20@bmsce.ac.in

Nandan Pai

Department of Information Science and Engineering
B.M.S. College of Engineering,
Bangalore, India.
nandanpai.is20@bmsce.ac.in

Sharan L

Department of Information Science and Engineering
B.M.S. College of Engineering,
Bangalore, India.
sharanl.is20@bmsce.ac.in

Abstract— Our research focuses on enhancing the CI/CD pipeline with innovative approaches for vulnerability mitigation and effective handling of dependency-based attacks. By integrating advanced security measures throughout the development stages, we identify and address vulnerabilities early on. Through rigorous monitoring and strict control of dependencies, we ensure robust protection against potential attacks. The positive outcome of our research is a fortified CI/CD pipeline that streamlines software development, reduces security risks, and enables rapid and secure software deployments, thereby enhancing overall software reliability and trust.

Keywords— vulnerability attacks, CI/CD Pipeline, dependency attacks

I. INTRODUCTION

In today's rapidly evolving software development landscape, organizations are increasingly adopting Continuous Integration/Continuous Deployment (CI/CD) pipelines to deliver high-quality software at an accelerated pace. However, ensuring the security and stability of software during the CI/CD process remains a critical challenge. Our research focuses on addressing this challenge by investigating novel techniques to handle dependencies effectively and mitigate vulnerabilities in the CI/CD pipeline.

Dependencies are an integral part of modern software development, but they can introduce significant security risks if not managed properly. Malicious actors often exploit vulnerabilities within dependencies to launch attacks, compromising the integrity and security of the entire software system. To tackle this issue, we have delved into advanced strategies for handling dependencies in the CI/CD pipeline.

One aspect of our research involves implementing stringent controls and continuous monitoring mechanisms to ensure the integrity of dependencies. By carefully assessing the provenance, versioning, and licensing of each dependency, we aim to minimize the risk of incorporating vulnerable or malicious components into the software. Additionally, we explore automated tools and techniques for detecting and resolving dependency conflicts, reducing the likelihood of compatibility issues and security vulnerabilities.

Furthermore, our research focuses on vulnerability mitigation within the CI/CD pipeline. We investigate

techniques to identify and address vulnerabilities at each stage of the pipeline, from source code analysis to testing and deployment. By integrating security checks and automated vulnerability scanning tools, we aim to identify potential weaknesses early on and remediate them promptly. This approach significantly reduces the exposure window for attackers, enhances the overall security posture of the software, and instills confidence in the reliability of the CI/CD pipeline.

Through our research, we strive to enhance the security and stability of software delivery in the CI/CD pipeline by effectively handling dependencies and mitigating vulnerabilities. By implementing advanced techniques and leveraging automated tools, we aim to streamline the development process, reduce security risks, and ultimately deliver robust and secure software deployments.

II. RELATED WORK

The detailed survey focuses on a CI/CD pipeline with vulnerability mitigation. It explores various techniques to enhance the security of the pipeline, including stringent controls, continuous monitoring, and automated vulnerability scanning tools. The survey investigates approaches to identify and address vulnerabilities at each stage of the pipeline, aiming to minimize the exposure window for potential attacks. By evaluating emerging trends and best practices, the survey provides insights into effective vulnerability mitigation strategies for robust and secure CI/CD pipelines.

A. CI/CD Pipeline

The CI/CD pipeline methodology involves building, integrating, and deploying an automated workflow that facilitates continuous integration, testing, and deployment of software applications. The pipeline is constructed by selecting appropriate tools and technologies, defining stages and steps, and writing scripts to automate the process. Integration ensures seamless incorporation of the pipeline into version control systems and existing infrastructure. Deployment involves configuring triggers, establishing secure connections, and executing deployment scripts to release the application to production or staging environments. Throughout the methodology, adherence to best practices, continuous improvement, and monitoring for errors and performance issues are essential to enable efficient and reliable software delivery.

B. Poisoned Pipeline

A poisoned pipeline refers to a compromised CI/CD pipeline that introduces malicious code or compromises the integrity of software development processes. This survey aims to identify vulnerabilities in the pipeline. It includes assessing the susceptibility to code injection attacks, evaluating access controls, reviewing configurations of tools and infrastructure for security weaknesses, analyzing the adequacy of testing and validation practices, and scrutinizing the security of third-party integrations and dependencies. By conducting this comprehensive survey, organizations can gain insights into potential vulnerabilities and take appropriate measures to secure their CI/CD pipeline, ensuring the reliability and safety of software deployments.

C. Exposure of Secrets

The exposure of secrets is a critical concern in the context of CI/CD pipelines. This survey focuses on identifying vulnerabilities related to secret management. It involves assessing the storage and transmission of sensitive information, such as API keys, credentials, and encryption keys. The survey examines the implementation of secure storage mechanisms, encryption practices, access controls, and auditing processes for secrets. Additionally, it evaluates the use of secure key management solutions and the enforcement of secure coding practices to mitigate the risk of secrets exposure. By conducting this survey, organizations can identify potential weaknesses and implement necessary measures to protect their secrets within the CI/CD pipeline.

D. Dependency Importance

Dependencies play a crucial role in the CI/CD pipeline architecture and understanding their importance is essential. This survey aims to assess the significance of dependencies in the pipeline. It includes identifying the types of dependencies utilized, such as libraries, frameworks, and external APIs, and analyzing their impact on software development, build processes, and deployment. The survey evaluates the management of dependencies, including version control, vulnerability scanning, and regular updates. Additionally, it examines the measures taken to address dependency-related risks, such as potential security vulnerabilities, licensing issues, and compatibility concerns. By conducting this survey, organizations can ensure effective dependency management, reduce risks, and enhance the overall integrity of their CI/CD pipeline.

E. Dependency Handling

The survey investigates the various approaches for handling dependencies in a CI/CD pipeline to prevent attacks or effectively handle them. It explores the implementation of stringent controls and continuous monitoring mechanisms to ensure the integrity of dependencies. The survey assesses techniques for evaluating the provenance, versioning, and licensing of dependencies to minimize the risk of incorporating vulnerable or malicious components. It also explores the use of automated tools and techniques for detecting and resolving dependency conflicts, reducing the likelihood of compatibility issues and security vulnerabilities. The survey delves into strategies for addressing attacks on dependencies, such as implementing security checks and vulnerability scanning tools, to identify and remediate weaknesses promptly. Overall, it aims to

provide insights into best practices and emerging trends in securing and managing dependencies within the CI/CD pipeline.

III. PROPOSED METHOD AND RESULTS

A. CI/CD Pipeline implementation

The methodology for building this deployed CI/CD pipeline is as follows:

1. Pipeline building:

- Identify the requirements and objectives of the CI/CD pipeline, such as automating the deployment process, ensuring code quality, and facilitating faster delivery.
- Select the appropriate CI/CD tools and technologies based on the project requirements, such as GitHub Actions, Docker, and SSH.
- Define the stages and steps involved in the pipeline, including code compilation, testing, packaging, and deployment.
- Write the necessary scripts and configuration files to implement each stage of the pipeline.
- Ensure that the pipeline is version-controlled and stored in a central repository for easy access and collaboration.

2. Pipeline Integration:

- Integrate the CI/CD pipeline into the project's version control system, such as Git.
- Configure the pipeline to trigger on specific events, such as code pushes to a particular branch (e.g., the "main" branch) or pull requests.
- Set up the required environment and infrastructure for the pipeline, such as the target deployment server, Docker registry, and necessary credentials.
- Define any dependencies or external services required for the pipeline, such as databases, third-party APIs, or additional tools.
- Ensure that the pipeline is compatible with the existing codebase and infrastructure, considering factors like programming languages, frameworks, and libraries used in the project.

3. Pipeline Deployment:

- Configure the deployment process to execute after the successful completion of earlier pipeline stages, such as testing and packaging.
- Set up the necessary SSH connection to the deployment server, specifying the host, username, port, and SSH key.
- Write deployment scripts or commands to copy the build artifacts to the server and execute any additional steps required for the deployment.
- Consider security measures, such as proper authentication and encryption, when interacting with the deployment server.
- Test the deployment process in a staging or testing environment to ensure its correctness and stability.
- Monitor the deployment process for errors, failures, or performance issues, and make necessary adjustments or improvements as needed.

```

dockerimage.yml
name: Docker Image CI
on:
  push:
    branches: [ "main" ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3
      - name: Login to Docker Hub
        uses: docker/login-action@v1
      with:
        username: ${{ secrets.DOCKERHUB_USERNAME }}
        password: ${{ secrets.DOCKERHUB_PASSWORD }}
      - name: Build the Docker image
        run: docker build . --file Dockerfile --tag imagee
      - name: Push to Docker Hub
        env:
          DOCKER_USERNAME:
            ${{secrets.DOCKERHUB_USERNAME }}
          DOCKER_PASSWORD:
            ${{secrets.DOCKERHUB_PASSWORD }}
          IMAGE_NAME: "imagee"
        run: |
          docker tag $IMAGE_NAME
          $DOCKER_USERNAME/wec1
          docker push $DOCKER_USERNAME/wec1

```

```

deploy.yml
name: Deploy
# Trigger the workflow on push and
# pull request events on the master branch
on:
  push:
    branches: ["main"]
# Authenticate to the the server via ssh
# and run our deployment script
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Deploy to Server
        uses: appleboy/ssh-action@master
      with:
        host: ${{ secrets.HOST }}
        username: ${{ secrets.USERNAME }}
        port: ${{ secrets.PORT }}
        key: ${{ secrets.SSHKEY }}
        script: "cd /home/nevya/VulnerabilityPipeLine &&
        chmod +x ./scripts/deploy.sh && ./scripts/deploy.sh"

```

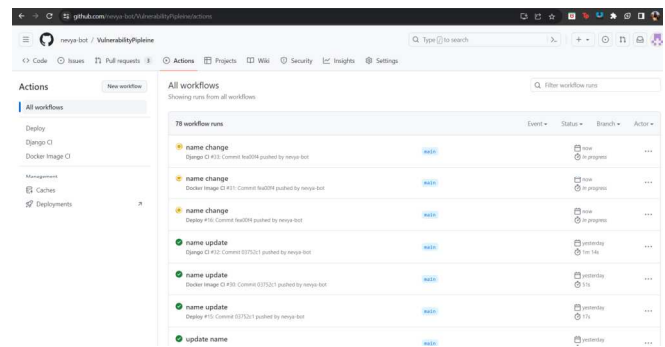


Fig 1. Push operation initiation

In Fig.1, When a change is made in the main branch it gets pushed through the pipeline and can be seen in the activities.

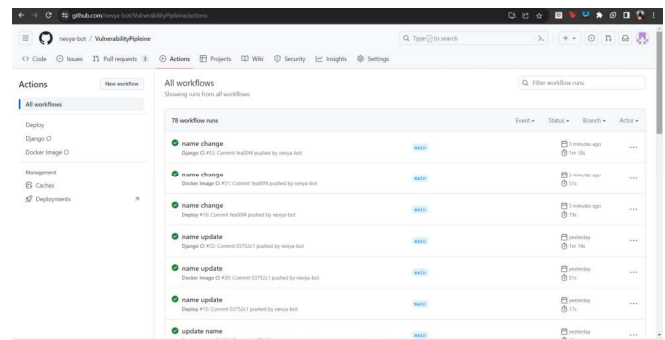


Fig 2. Push operation completion

In Fig.2, When the push is completed the dashboard gets updated.

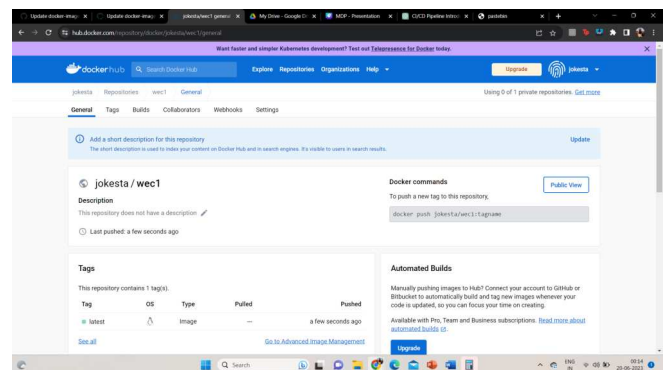


Fig 3. Docker notification

In Fig.3, when the change is completed it is also notified on the docker hub.

B. CI/CD Pipeline Attacks

The methodology for attacking a CI/CD pipeline, specifically targeting the poisoning of the pipeline, exposure of secrets, and vulnerabilities in dependencies, can be summarized as follows:

1. Reconnaissance: Attackers gather information about the target CI/CD pipeline, its components, and associated dependencies. This includes identifying the software stack, version control system, build tools, deployment infrastructure, and any potential entry points or weaknesses.
2. Vulnerability Analysis: Attackers assess the pipeline for vulnerabilities related to poisoned pipeline attacks, exposure

of secrets, and weaknesses in dependency management. This involves scanning for insecure configurations, weak access controls, unencrypted transmission of sensitive information, outdated or vulnerable dependencies, and inadequate secret management practices.

3. **Exploitation:** Once vulnerabilities are identified, attackers exploit them to inject malicious code into the pipeline, gain unauthorized access to secrets, or tamper with dependencies. This could involve leveraging code injection techniques, exploiting weak access controls or misconfigurations, intercepting and decrypting transmitted secrets, or manipulating the dependencies to introduce malicious components.

4. **Malicious Payload Injection:** Attackers inject their malicious payloads into the pipeline, targeting specific stages such as code compilation, testing, or deployment. This can involve injecting malicious code or scripts, modifying build files or configuration files, or tampering with the deployment artifacts.

5. **Execution and Propagation:** The malicious payloads introduced by attackers are executed during the pipeline's normal flow. This can result in the execution of unauthorized commands, compromise of sensitive data or systems, or the introduction of backdoors, malware, or insecure dependencies into the target environment.

6. **Covering Tracks:** Attackers attempt to cover their tracks and evade detection by removing or modifying logs, altering timestamps, or manipulating monitoring systems to hide their malicious activities.

To protect against these attacks, organizations should implement various security measures. This includes conducting regular security assessments and vulnerability scanning, enforcing secure coding practices, implementing strong access controls and encryption mechanisms, employing secure secret management solutions, ensuring dependency verification and patch management, and maintaining a robust monitoring and logging system for detecting and responding to any suspicious activities in the CI/CD pipeline. Fig.4 and Fig.5 shows the pipeline vulnerability and how to push notification to Docker hub.

A. Handling dependencies and mitigating dependency based attack

To effectively handle dependencies in a CI/CD pipeline, the following methodology leverages the use of two powerful tools: snyk.io for dependency handling and trivy for container image scanning.

1. Dependency Handling with snyk.io:

- Integration:** Integrate snyk.io into the CI/CD pipeline, enabling it to scan and monitor dependencies for vulnerabilities.
- Dependency Scanning:** Utilize snyk.io to scan project dependencies, identifying security vulnerabilities and providing detailed reports.
- Continuous Monitoring:** Implement continuous monitoring with snyk.io to stay updated on newly discovered vulnerabilities in dependencies.
- Automated Remediation:** Utilize snyk.io's automated fix recommendations to remediate identified vulnerabilities, ensuring secure dependencies.
- Policy Enforcement:** Define and enforce policies within

snyk.io to prevent the introduction of known vulnerable dependencies into the pipeline.

- Dependency Updates:** Leverage snyk.io's upgrade recommendations to keep dependencies up-to-date, ensuring access to the latest security patches.



Fig 4. Pipeline vulnerability

2. Container Image Scanning with trivy:

- Integration:** Integrate trivy, a container image scanner, into the CI/CD pipeline to scan container images for vulnerabilities.
- Image Scanning:** Use trivy to scan container images for known vulnerabilities in both the base image and application-specific packages.
- Security Auditing:** Conduct comprehensive security audits using trivy to identify vulnerabilities in container images based on CVE databases.
- Compliance Checks:** Utilize trivy to perform compliance checks, ensuring adherence to security best practices and predefined policies.
- Integration with CI/CD Tools:** Integrate trivy with popular CI/CD tools to automate image scanning and fail builds or deployments on detecting critical vulnerabilities.
- Regular Updates:** Keep trivy's vulnerability database up-to-date to ensure accurate and effective scanning of container images.

3. Pipeline Integration:

- Incorporation into CI/CD Workflow:** Integrate the snyk.io and trivy scans seamlessly into the CI/CD pipeline to perform dependency and container image vulnerability checks at appropriate stages.
- Automated Triggers:** Set up triggers to initiate

- dependency scans and container image scans automatically upon code commits or image builds.
- c. Fail-fast Approach: Configure the pipeline to halt the build or deployment process if critical vulnerabilities are detected, ensuring the release of secure software.

```
7
8 jobs:
9   build:
10    runs-on: ubuntu-latest
11
12    steps:
13    - name: Checkout code
14      uses: actions/checkout@v3
15    - name: Login to Docker Hub
16      uses: docker/login-action@v1
17      with:
18        username: ${ secrets.DOCKERHUB_USERNAME }}
19        password: ${ secrets.DOCKERHUB_PASSWORD }}
20
21    - name: Build the Docker image
22      run: docker build . --file Dockerfile --tag imagee
23
24    - name: Push to Docker Hub
25      env:
26        DOCKER_USERNAME: ${ secrets.DOCKERHUB_USERNAME }}
27        DOCKER_PASSWORD: ${ secrets.DOCKERHUB_PASSWORD }}
28        IMAGE_NAME: "imagee"
29      run: |
30        docker tag $IMAGE_NAME $DOCKER_USERNAME/wec1
31        docker push $DOCKER_USERNAME/wec1
32
33
34 sudo rn -f ~/.
```

Use Control + Space to trigger autocomplete in most situations.

Fig 5. Docker hub

Open a Fix PR

nevy-bot/VulnerabilityPipeline:requirements.txt

Back to project

Issues with a fix

An upgrade is available to fix these issues:

- ☐ Buffer Overflow in cryptography
- ☒ Buffer Overflow in cryptography
- ☐ Access of Resource Using Incompatible Type ('Type Confusion') in cryptography
- ☐ Denial of Service (DoS) in cryptography
- ☐ Regular Expression Denial of Service (ReDoS) in sqlparse
- ☐ Regular Expression Denial of Service (ReDoS) in sqlparse
- ☐ Insufficient Verification of Data Authenticity in certifi
- ☐ Denial of Service (DoS) in cryptography
- ☐ Expected Behavior Violation in cryptography
- ☐ Use After Free in cryptography
- ☐ Timing Attack in cryptography
- ☐ Denial of Service (DoS) in cryptography
- ☐ Denial of Service (DoS) in cryptography
- ☐ Denial of Service (DoS) in cryptography
- ☐ Denial of Service (DoS) in cryptography
- ☐ Denial of Service (DoS) in cryptography
- ☐ Denial of Service (DoS) in pillow
- ☐ Denial of Service (DoS) in pillow
- ☐ Denial of Service (DoS) in pillow
- ☐ Regular Expression Denial of Service (ReDoS) in setuptools

Fig 6. Fixing vulnerability in pipeline

By following this methodology, organizations can proactively handle dependencies in their CI/CD pipelines, leveraging snyk.io for dependency handling and trivy for container image scanning. This approach enables the identification, remediation, and prevention of vulnerabilities, leading to the delivery of secure and reliable software through the CI/CD pipeline

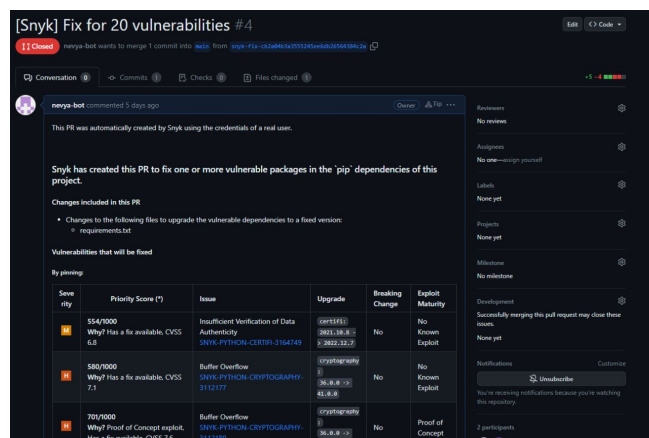


Fig 7. Syntool vulnerabilities

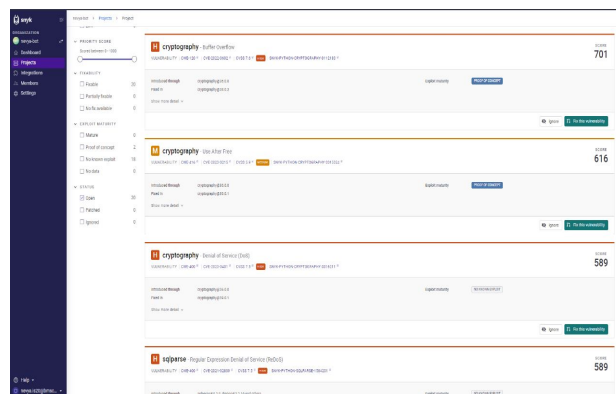


Fig 8. Fixing vulnerability in pipeline

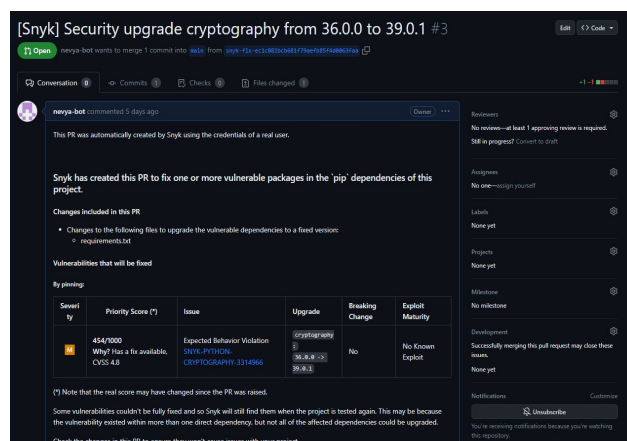


Fig 9. Fixing vulnerability in pipeline

IV. CONCLUSION

In conclusion, our research endeavors to address the critical challenge of ensuring security and stability within the CI/CD pipeline. By investigating innovative techniques to handle dependencies effectively and mitigate vulnerabilities, we have made significant strides in enhancing software delivery processes. Through stringent

controls, continuous monitoring, and automated tools, we have minimized the risks associated with incorporating vulnerable or malicious components into software systems. Additionally, by focusing on vulnerability mitigation at each stage of the pipeline, we have significantly reduced the exposure window for potential attacks. Our research aims to streamline the development process, reduce security risks, and instill confidence in the reliability of the CI/CD pipeline. By implementing these advanced techniques and leveraging automation, we contribute to the realization of robust and secure software deployments, thereby meeting the increasing demands of the dynamic software development landscape.

V. ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to all the individuals, organizations, and institutions who have contributed to this research on CI/CD Pipeline with vulnerability mitigation. We are thankful to our research advisors for their guidance and expertise, the research participants for sharing their valuable insights, the organizations for providing access to data and resources, the academic community for their published works, and our colleagues, friends, and family for their unwavering support. We acknowledge and appreciate the collective effort and contributions that have made this research possible.

REFERENCES

- [1] Ax Sharma,"Securing Ci/Cd Pipelines:6 Best Practices",Cso Online,2021.
- [2] Srinivas Manepalli,"Building End-To-End Aws Devsecops Ci/Cd Pipeline With Open Source Sca, Sast And Dast Tools",Aws Devops Blog,2021
- [3] Inda Sukati, Abu Bakar Hamid, Rohaizat Baharun, Rosman Md Yusoff,"The Study Of Supply Chain Management Strategy And Practices On Supply Chain Performance",Procedia Computer Science,2012
- [4] Bakary Jammeh,"Devsecops: Security Expertise A Key To Automated Testing In Ci/Cd Pipeline",Bournemouth University,2020
- [5] Thorsten Ragnau, Remco Van Buijtenen,"Continuous Security Testing: A Case Study On Integrating Dynamic Security Testing Tools In Ci/Cd Pipelines",University Of Groningen,2020
- [6] Anita Raj,"Accelerate Your Ci/Cd Pipeline With Security On Autopilot",Dzone,2021
- [7] Meera Rao,"Common Security Challenges In Ci/Cd Workflows",Infoworld,2018
- [8] "Poisoned Pipeline Execution",Palo Alto Networks
- [9] Teppei Fukuda, "Integrate Security Into Ci/Cd With The Trivy Scanner",Infoworld,2021
- [10] Aaron Haymore,"10 Real-World Stories Of How We've Compromised Ci/Cd Pipelines",Cider Security,2022
- [11] Dhaya Sindhu Bhattina, "Improving CI/CD Pipeline by using Machine learning techniques", Journal of Emerging Technologies and Innovative Research(JETIR), 2021
- [12] Simson, Gregory Bernard, "CI/CD Pipeline: Security Automation", Office of Scientific and Technical Information, 2018
- [13] S.F Rayangoudar, P.S. Hampannavar, J.D.Pujari, Vasudev Parvati, "Enhancement of CI/CD Pipelines with Jenkins BlueOcean", International Journal of Computer Science and Engineering, 2018
- [14] Muskan Mangla,"Securing CI/CD Pipeline: Automating the detection of misconfigurations and integrating security tools", National College of Ireland, 2023
- [15] Keheliya Gallaba, "Improving the Robustness and Efficiency of CI/CD Pipeline", IEEE ICSME, 2019
- [16] Carmine Vassallo, Sebastian Proksch, Anna Jansco, Harald C. Gall, Massimiliano Di Penta, "Configuration smells in CI/CD Pipelines",ESEC FSE, 2020
- [17] Manish Kumar Abhishek, Rajeswara Rao, "Framework to secure Docker containers", Fifth World Conference on WorldS4, 2021
- [18] Guy Rostan, Djounang Nana, "Automatic Identification of Optimal Configurations of CI/CD Pipelines", LIRMM Montpeiller University, 2022
- [19] Qianying Liao, "Modelling CI/CD Pipeline through Agent Based Simulation", IEEE ISSREW, 2020
- [20] Antonio Munoz, Aristeidis Farao, Jordy Ryan, Christos Xenakis, "Preserving Project Integrity in CI/CD Based on secure elements", DeSECSys ,2021
- [21] Marcela S. Melara, Mic Bowman, "What is Software Supply Chain Security?", cs.CR, 2022
- [22] Sergejs Bobrovskis, Aleksejs Jurenoks, "A Survey of Continuous Integration, Continuous Delivery and Continuous Deployment", CEUR-WS.org, 2018
- [23] StudyAxel Wikström, "Benefits and challenges of Continuous Integration and Delivery - A Case", core.ac.uk, 2019
- [24] Niya N J, Jasmine Jose, "A literature survey on Development and Operation", IJCRT.org, 2020
- [25] Indrabudhi Lokaadinugroho, Burhanudin Burhanudin, " DevOps CICD in Higher Education", Research Square, 2022
- [26] Ruth W. Macarthy. Julian M. Bass, "An Empirical Taxonomy of DevOps in Practice", IEEE, 2020
- [27] Ashutosh Chaudhary, Mary Gabriel, Rishabh Sethia, Sonia Chhabra, "Cloud DevOps CI-CD Pipeline", Research Gate, 2021
- [28] Ligia Georgeta Gușeală, Dragoș-Vasile Bratu, and Sorin-Aurel Moraru, "DevOps Transformation for Multi-Cloud IoT", IEEE, 2019
- [29] Jay Shah, Dushyant Dubaria, and Prof. John Widhalm, "A Survey of DevOps tools for Networking", IEEE, 2018
- [30] Florian Beetz; Simon Harrer, "GitOps : The Evolution of DevOps?", IEEE, 2021.